Low-Level Design (LLD)

User Payout Management System

SDE Intern Assignment — System Design (LLD)

Advance payouts · Reconciliation-driven final payouts · Withdrawal cooldown · Failed-payout recovery

Table of Contents

1. Scope

This document describes the low-level design of the User Payout Management System: a system that logs affiliate sales, pays users a 10% advance on pending sales, reconciles sales into approved/rejected states to compute final payouts, and lets users withdraw their wallet balance subject to a 24-hour cooldown and asynchronous gateway settlement.

2. High-Level Architecture

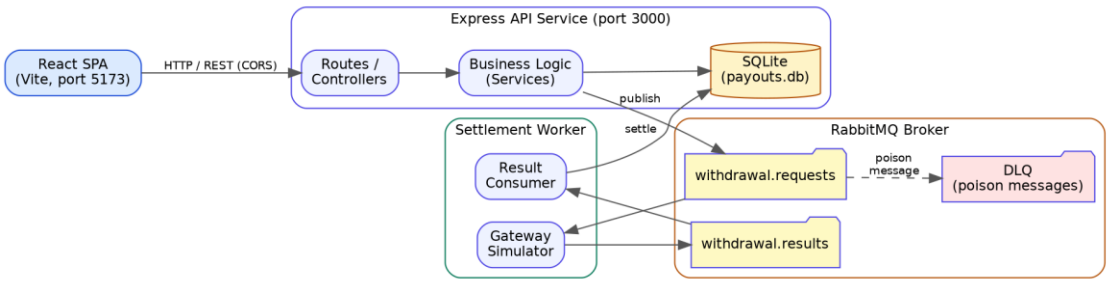

Figure 1 — System architecture: React SPA, Express API, SQLite, RabbitMQ broker, and the async settlement worker.

Component responsibilities

- API Service: stateless Express app; all writes go through a single SQLite connection guarded by transactional locking (BEGIN IMMEDIATE).
- Worker Service: standalone Node process, decoupled from the API so a slow or failing gateway never blocks user-facing requests.
- Broker: RabbitMQ with a Dead Letter Queue for messages that repeatedly fail processing, so they are not silently dropped or endlessly retried.
- Offline mode: QUEUE_DISABLED=true swaps RabbitMQ for an in-process synchronous call, so the whole system (including tests) runs without external infrastructure.

3. Database Schema (Entity Relationship Diagram)

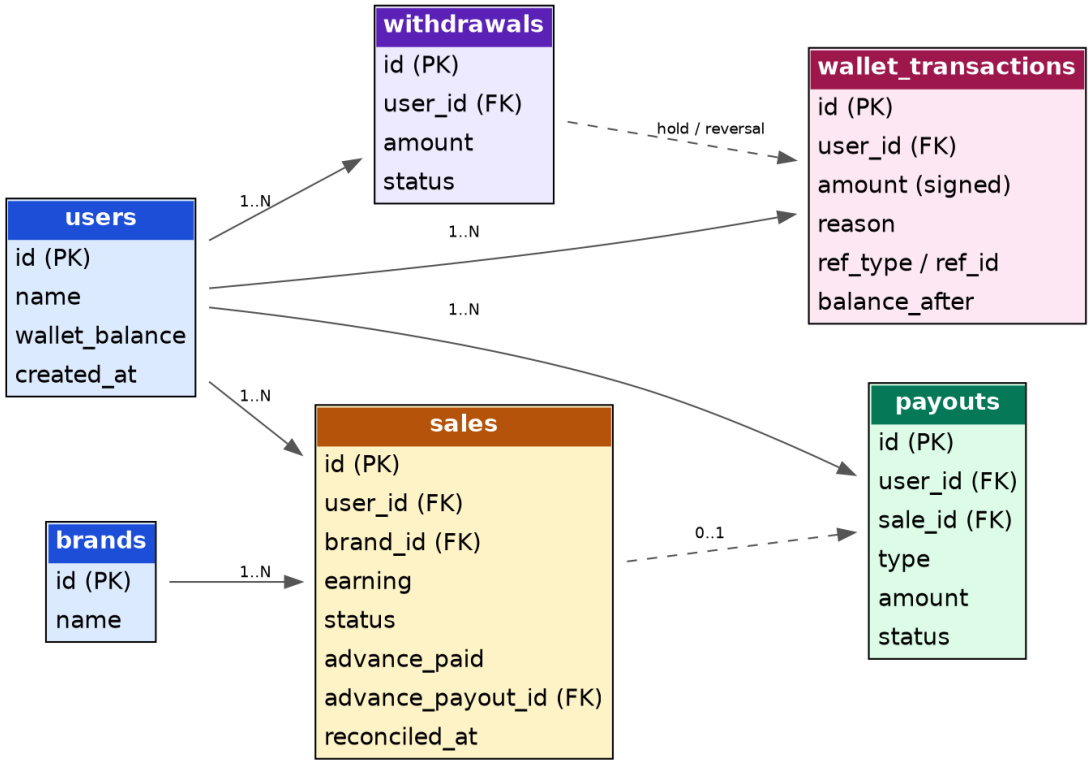

Figure 2 — Entity-relationship diagram for users, brands, sales, payouts, wallet_transactions, and withdrawals.

Key relationships

- A user has many sales, payouts, wallet_transactions, and withdrawals.
- Each sale optionally references the payout that advanced it (sales.advance_payout_id → payouts.id) — this is the structural idempotency key described in Section 6.
- wallet_transactions is an append-only audit ledger; it references withdrawals/payouts polymorphically via ref_type + ref_id.

Full DDL, indexes, and money/precision notes are provided in the companion document DATABASE_SCHEMA.md.

4. Class Design

The backend follows a layered repository-service pattern: controllers parse HTTP, services own business rules, repositories own SQL. No layer skips another.

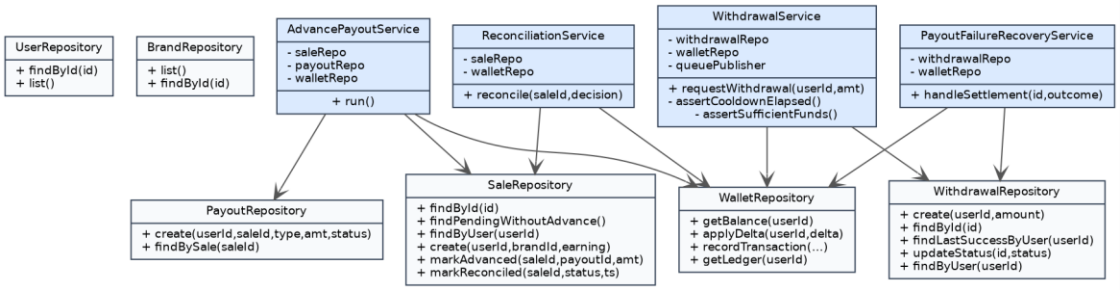

Figure 3 — Repository and service class map.

Repository layer

Class	Key methods	Notes
UserRepository	findById, list	Read-mostly; balance mutations go through WalletRepository only.
BrandRepository	list, findById	Static reference data.
SaleRepository	findPendingWithoutAdvance, markAdvanced, markReconciled	findPendingWithoutAdvance enforces advance-payout idempotency at the SQL level.
PayoutRepository	create, findBySale	Immutable records of every payout event.
WalletRepository	applyDelta, recordTransaction, getLedger	The only class permitted to write users.wallet_balance.
WithdrawalRepository	findLastSuccessByUser, updateStatus	findLastSuccessByUser filters status='SUCCESS' only — makes cooldown-voiding automatic.

Service layer

AdvancePayoutService.run()

- Open BEGIN IMMEDIATE transaction.
- SaleRepository.findPendingWithoutAdvance().
- For each sale: advance = round(earning × 0.10, 2); create ADVANCE payout; mark sale advanced; credit wallet; record ledger entry.
- Commit. Returns processed count and total advanced.

ReconciliationService.reconcile(saleId, decision)

- Load sale row inside the transaction (implicit row lock via BEGIN IMMEDIATE).
- Guard: sale must exist and be pending, else 404 / 409.
- approved → adjustment = earning − advance_paid. rejected → adjustment = −advance_paid.
- Mark reconciled, apply wallet delta (may go negative on rejection), record ledger entry, commit.

WithdrawalService.requestWithdrawal(userId, amount)

- assertCooldownElapsed — reject with 429 if < 24h since the user's last SUCCESS withdrawal.
- assertSufficientFunds — reject with 400 if amount exceeds balance.

- Debit the wallet immediately (hold), insert PENDING withdrawal, record ledger entry, commit.
- Publish to withdrawal.requests only after commit, so a queue failure never leaves an uncommitted hold.

PayoutFailureRecoveryService.handleSettlement(withdrawalId, outcome)

- SUCCESS → mark SUCCESS; hold already debited, nothing further to do.
- FAILED / REJECTED / CANCELLED → mark status, credit the amount back, record a WITHDRAWAL_REVERSAL ledger entry. No separate cooldown-void step is needed.

5. Sequence Diagrams

5.1 Advance Payout Job

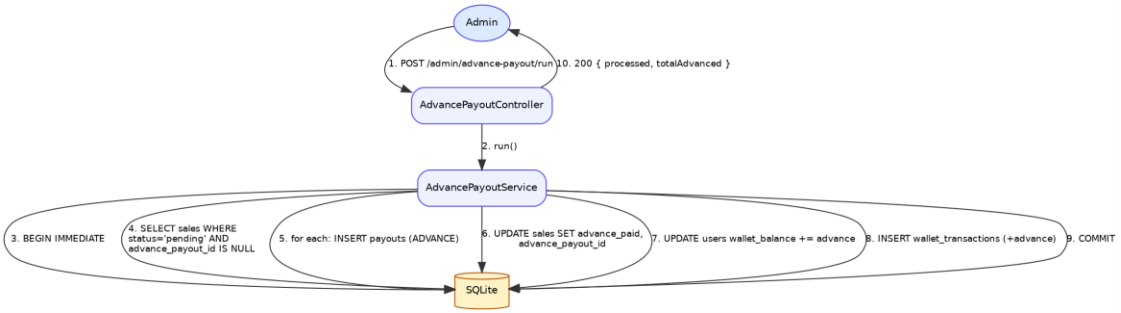

Figure 4 — Advance payout job flow.

Idempotency guarantee: the WHERE advance_payout_id IS NULL predicate is the sole source of truth for “has this sale already been advanced?” Running the job twice, or concurrently, cannot double-pay a sale because the second transaction's SELECT will no longer see rows already claimed by the first.

5.2 Reconciliation (Approve / Reject)

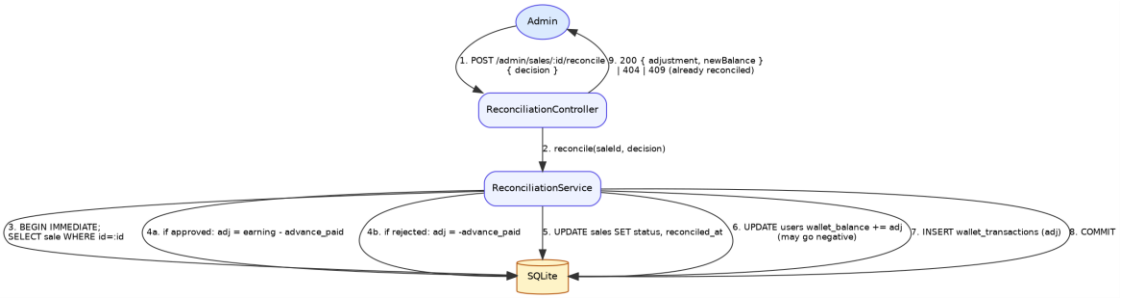

Figure 5 — Reconciliation flow for approved and rejected sales.

5.3 Withdrawal Request

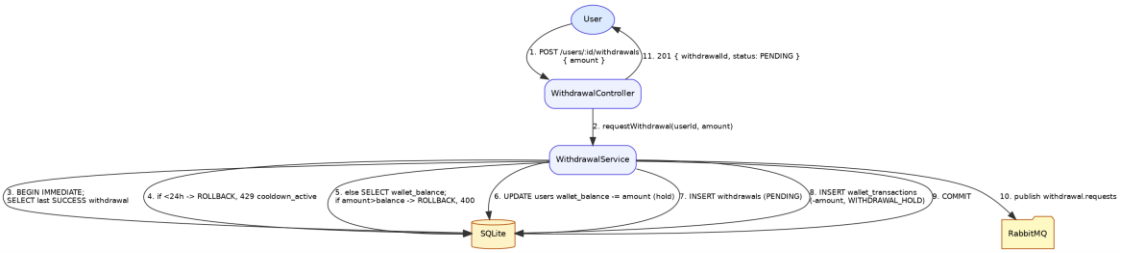

Figure 6 — Withdrawal request flow, including cooldown and balance checks.

5.4 Failed / Cancelled / Rejected Payout Recovery

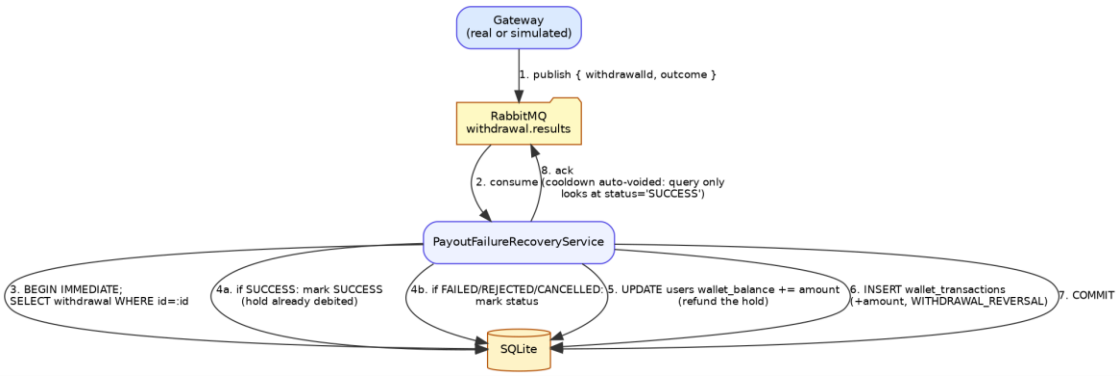

Figure 7 — Settlement callback handling and reversal flow.

Cooldown-voiding logic: the 24h cooldown check only ever looks at withdrawals with status = 'SUCCESS'. Because a reversed withdrawal is stamped FAILED / REJECTED / CANCELLED rather than SUCCESS, it is automatically invisible to the cooldown query — no separate “void” flag or extra write is required.

6. State Machines

6.1 Sale Status

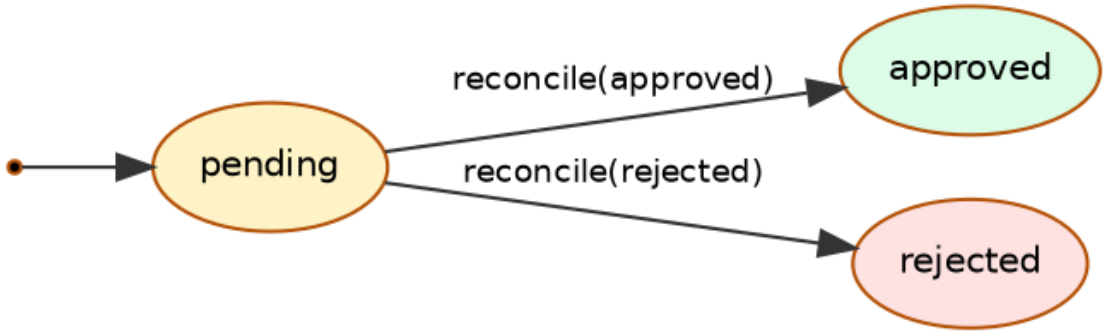

Figure 8 — Sale status lifecycle.

6.2 Withdrawal Status

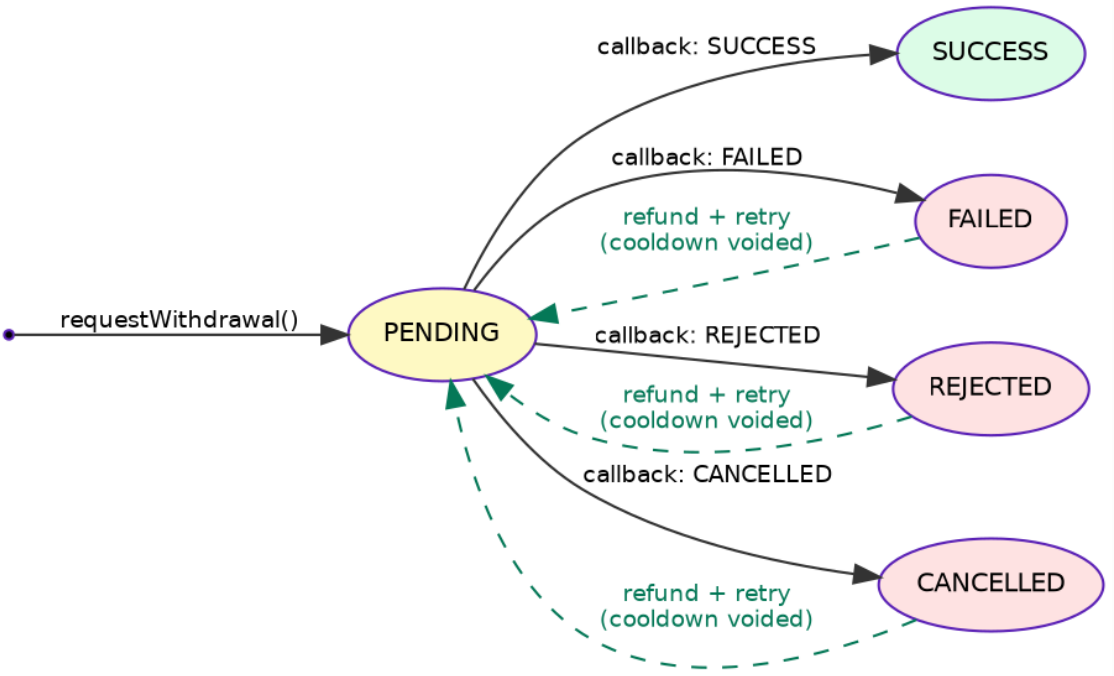

Figure 9 — Withdrawal status lifecycle, including the recovery/retry path.

7. API Reference (Summary)

Full request/response examples for every endpoint are in the companion document API_DOCUMENTATION.md. Summary:

Method	Route	Description
GET	/health	Service health check
GET	/users	List seeded users
GET	/brands	List brands
POST	/sales	Log a new pending sale
GET	/sales	List sales (filterable by user)
POST	/admin/advance-payout/run	Run the advance payout job
POST	/admin/sales/:id/reconcile	Approve or reject a sale
GET	/users/:userId/balance	Fetch withdrawable balance
GET	/users/:userId/transactions	Fetch wallet ledger
GET	/users/:userId/withdrawals	List withdrawals
POST	/users/:userId/withdrawals	Request a withdrawal
POST	/withdrawals/:id/status	Simulate a gateway settlement callback

8. Edge Cases & Failure Handling (Summary)

Full edge-case analysis is in the companion document EDGE_CASES.md. Highlights:

Area	Edge case	Handling
Advance payout	Job run twice / concurrently	advance_payout_id IS NULL predicate prevents double payment
Reconciliation	Reconciling an already-reconciled sale	409 Conflict, idempotent no-op
Reconciliation	Rejection pushes balance negative	Allowed by design; nets out against future approved sales
Withdrawal	Request within 24h of last SUCCESS	429 cooldown_active with retryAfter
Withdrawal	Amount exceeds balance	400 insufficient_funds, no DB write
Recovery	Duplicate settlement callback	Idempotent — only mutates rows currently PENDING
Recovery	FAILED/REJECTED/CANCELLED outcome	Full refund, ledger reversal entry, cooldown auto-voided
Concurrency	Two writers touch the same wallet at once	BEGIN IMMEDIATE serializes writers
Money	Floating-point drift	Rounded to 2 decimals at every write

9. Key Design Decisions & Trade-offs

BEGIN IMMEDIATE instead of application-level locks

SQLite's default BEGIN DEFERRED only takes a write lock at the first write, leaving a TOCTOU race window. BEGIN IMMEDIATE acquires the write lock up front, serializing writers. Trade-off: serializes all writes system-wide — acceptable at this scale; would move to row-level locks in Postgres/MySQL for higher throughput.

Denormalized wallet_balance + append-only ledger

Balance is cached for fast reads; wallet_transactions is the source of truth for audit and drift-detection. Both are written in the same transaction on every code path.

Structural idempotency, not a distributed lock

advance_payout_id IS NULL is both the eligibility filter and the idempotency check — there is no separate check-then-act race to guard against.

Cooldown keyed only on status = 'SUCCESS'

Makes “void the cooldown on failure” free: a reversed withdrawal was never a SUCCESS row, so it never started a cooldown. Trade-off: a user could have multiple PENDING withdrawals in flight, bounded only by available balance — accepted as reasonable and consistent with real payment rails.

Async settlement via queue, not a synchronous gateway call

Avoids blocking requests on slow gateways and decouples gateway outages from API availability. Justified specifically because failed/cancelled/rejected payout recovery is a first-class requirement.

Currency as rounded REAL, not integer minor-units

Simpler for this assignment's scope; rounds to 2 decimals on every write. Production systems handling real money should use integer paise instead to eliminate floating-point risk entirely.

Additional decisions, alternatives considered, and why they were rejected are documented in full in DESIGN_DECISIONS.md.